One Codebase, Many Outputs

E2E Testing as a Platform in Open edX

The Problem

- Frontend code has a **short shelf life**
- Screenshots go stale within a single release cycle
- Accessibility audits happen quarterly (if you're lucky)
- Visual bugs slip through functional tests

The Insight

> The parts of Open edX worth **documenting** are almost always the parts worth writing **E2E tests** for.

So why maintain two separate efforts?

One Codebase, Four Outputs

1. ✅ **Test Coverage** - Validate critical user flows
2. 📚 **Documentation** - Auto-generated user guides with screenshots
3. ♿ **Accessibility Reports** - WCAG compliance with actionable tickets
4. 🎨 **Visual Regression** - Pixel-level diff detection with red highlights

Demo Setup

The Project Structure

```
openedx-e2e-tests/
├── tests/
│   ├── auth/                       # Authentication tests
│   ├── courses/                    # Course management tests
│   ├── testdoc/                    # Markdown-driven test docs
│   ├── common/
│   │   ├── page-objects.ts
│   │   ├── a11y-helpers.ts
│   │   └── visual-regression-helpers.ts
│   └── __visual-baselines__/       # Baseline images for visual regression
├── utils/
│   ├── testdoc.ts                  # Documentation framework
│   ├── element-highlighter.ts      # UI element highlighting
│   ├── markdown-test-parser.ts     # Markdown test parser
│   └── dates.ts                    # Date utilities
├── artifacts/                      # All outputs (gitignored)
│   ├── testdoc-output/             # Auto-generated docs
│   ├── a11y-reports/               # Accessibility reports
│   ├── visual-regression/          # Current screenshots & diffs
│   ├── playwright-report/          # Test execution reports
│   ├── test-results/               # Test result artifacts
│   └── downloads/                  # Downloaded files from tests
├── docs/                           # Project documentation
└── scripts/                        # Setup & utility scripts
```

Getting Started

```
npm install
npm run install:browsers
npm run setup                # creates test users
npm test                     # runs all tests
```

Test data:

- Regular user: `testuser` / `password123`
- Admin user: `adminuser` / `admin123`
- Demo course: `course-v1:edX+DemoX+Demo_Course`

From Vanilla Playwright to Platform

Vanilla Playwright Test

```
test('Learner dashboard displays enrolled courses', async ({ page }) => {
  await page.goto('/dashboard');
  await expect(page).toHaveURL(/\/login/);

  await page.fill('input[name="emailOrUsername"]', 'testuser');
  await page.fill('input[name="password"]', 'password123');
  await page.click('button[name="sign-in"]');

  await expect(page).toHaveURL(/\/dashboard/);
  const courseCards = page.locator('.course-card');
  await expect(courseCards).toHaveCountGreaterThan(0);
});
```

What you get: Test coverage ✅

Add Testdoc for Documentation

```
test('Learner Dashboard Walkthrough', async ({ page }) => {
  const doc = new TestdocTest(page, 'learner-dashboard', {
    title: 'Navigating the Learner Dashboard',
    overview: 'This guide walks through the learner dashboard...',
  });

  await doc.fill('input[name="emailOrUsername"]', 'testuser',
    'Enter Username', 'Enter your registered username.');

  await doc.click('button[name="sign-in"]', 'Submit Login',
    'Click the sign-in button to authenticate.');
});
```

What you get: Test coverage ✅ + Documentation 📚

How Testdoc Works

1. **Highlight** the element with orange outline + box shadow
2. **Wait** 500ms for CSS to render
3. **Screenshot** the highlighted element (full page or cropped)
4. **Remove** the highlight
5. **Perform** the action (click, fill, etc.)

Output: `artifacts/testdoc-output/learner-dashboard/`

- Numbered steps
- Annotated screenshots
- Prerequisites & related topics

Markdown-Driven Authoring

Documentation First, Test Second

Write Documentation, Get Tests

```
## Enter Your Credentials

Click on the email field and enter your login identifier.

```testdoc
await testdoc.fill({
  selector: 'input[name="emailOrUsername"]',
  value: 'testuser',
  title: 'Enter your email or username',
  description: 'Type your login identifier',
  elementOnly: 'form[id="sign-in-form"]',
  padding: 30
});
```
```

Why This Matters

Traditional workflow:

1. Engineer builds feature ➡️ writes test ➡️ ships
2. Someone writes docs separately
3. UI changes ➡️ docs become stale

Markdown-driven workflow:

1. Documentation **IS** the test
2. UI changes ➡️ test fails
3. Fix selector ➡️ docs update automatically

Accessibility: Actionable Stories

Running Axe During Tests

```
import { assertA11y } from '../common/a11y-helpers';

test('dashboard accessibility', async ({ page }, testInfo) => {
  await page.goto('/dashboard');

  await assertA11y(page, {
    warnOnly: true,         // don't fail the test
    report: true,           // generate HTML report
    reportName: 'dashboard'
  }, testInfo);
});
```

Reports land in `artifacts/a11y-reports/`

From JSON Blob to Jira Ticket

Raw Axe output:

```
{
  "id": "color-contrast",
  "impact": "serious",
  "nodes": [...]
}
```

Our transformation:

1. Screenshot each violation with **red highlight**
2. Categorize by WCAG criterion
3. Generate story-ready description with impact + fix

Each violation = one assignable ticket with visual evidence

Visual Regression Testing

Pixel-Level Confidence

The Problem Visual Regression Solves

In Open edX micro-frontends:

- Component renders ✅
- API returns data ✅
- **But layout broke** because someone updated a shared CSS variable ❌

Functional tests can't catch this.

Our Custom Visual Regression Utility

```
import { VisualRegression } from '../common/visual-regression-helpers';

test('account settings visual regression', async ({ page }, testInfo) => {
  const vr = new VisualRegression(page, testInfo);

  await page.goto('/account/');

  await vr.captureAndCompare({
    name: 'account-page',
    fullPage: true,
    threshold: 0.15,
    mask: ['.timestamp', '.last-login-time']
  });
});
```

How It Works

First run:

- Creates baseline → `tests/__visual-baselines__/chromium/.../account-page.png`
- ✅ Tracked in git

Subsequent runs:

- Captures current screenshot
- Compares pixel-by-pixel using `pixelmatch`
- Generates **red-highlighted diff** if pixels changed

Red-Highlighted Diffs

Diff image shows:

- Gray pixels = unchanged
- Bright red pixels = changed (major)
- Light red pixels = subtle differences

Output:

- `artifacts/visual-regression/.../current/account-page.png`
- `artifacts/visual-regression/.../diff/account-page-diff.png`

Creating & Updating Baselines

Initial baseline:

```
npm run test tests/auth/login.spec.ts -- --project=chromium
git add tests/__visual-baselines__/
git commit -m "Add visual regression baseline"
```

Update after intentional change:

```
rm -rf tests/__visual-baselines__/chromium/your-test-name/
npm run test tests/auth/login.spec.ts -- --project=chromium
git add tests/__visual-baselines__/
```

Stability: What Makes Tests Fragile

❌ **Non-deterministic content** - timestamps, random data
❌ **Animations** - screenshots at different moments
❌ **Font rendering** - different across OS/browser versions
❌ **Network timing** - assets not fully loaded
❌ **Viewport inconsistency** - rely on explicit dimensions

✅ **Solution:** Disable animations, mask dynamic content, wait for stability

Running in the Pipeline

CI/CD Workflow

```yaml
- name: Run E2E Tests
  env:
    BASE_URL: ${{ secrets.STAGING_URL }}
  run: |
    npm run setup
    npm test
    npm run testdoc

- name: Upload Test Artifacts
  uses: actions/upload-artifact@v3
  if: always()
  with:
    name: test-artifacts
    path: artifacts/
```

Artifacts as Review Context

Reviewers download `artifacts/` and get:

- 📊 **Test results** - Playwright HTML report with traces
- 📚 **Documentation** - Auto-generated user guides in `testdoc-output/`
- ♿ **Accessibility reports** - Violations with highlighted screenshots
- 🎨 **Visual diffs** - Red-highlighted pixel changes

No need to pull the branch and run tests locally!

When PR Merges

1. Documentation publishes to docs site
2. Visual regression baselines update (if approved)
3. New baselines become source of truth

Continuous validation:

- Same tests run nightly against staging
- Catch regressions before production

AI-Assisted Test Completion

The Gap

Right now, you still need to manually:

- Add Testdoc calls to vanilla Playwright
- Write documentation prose around Testdoc
- Add `assertA11y()` checks
- Add visual regression baselines
- Fill in selectors for markdown-driven tests

Each layer is more work on top of the last

Claude Code + CLAUDE.md

CLAUDE.md teaches Claude Code about your project:

- Testdoc API conventions
- Page object locations
- Accessibility helper usage
- Visual regression patterns
- Test data credentials

Claude reads this once at session start

AI Completes Whatever You Start

Starting from vanilla Playwright:

→ Generates complete markdown-driven version with prose, testdoc blocks, a11y checks, visual regression

Starting from Testdoc:

→ Generates markdown wrapper with context, tips, troubleshooting

Starting from markdown:

→ Fills in executable testdoc blocks with correct selectors from page objects

Custom Slash Commands

```
<!-- .claude/commands/testdoc.md -->
Read the file at $ARGUMENTS and determine its current state:

- If vanilla Playwright: generate complete markdown-driven testdoc
- If Testdoc .spec.ts: generate markdown wrapper
- If .md with prose only: add testdoc code blocks

In all cases:
- Use object config for testdoc methods
- Include assertA11y checks after page navigation
- Add visual regression with captureAndCompare()
- Write for reader who has never used Open edX
```

Usage: `/testdoc tests/auth/login.spec.ts`

The Platform Vision

One Test Suite, Four Outputs

```
PR opened
  ↓
Test data seeded
  ↓
E2E tests run
  ├─→ Testdoc captures annotated screenshots
  ├─→ assertA11y generates accessibility reports
  └─→ Visual regression compares pixel-by-pixel
      ↓
Artifacts uploaded to PR
  ↓
Reviewer downloads artifacts
  ├─→ Test reports
  ├─→ Documentation
  ├─→ A11y violations
  └─→ Visual diffs (red highlights)
```

The Payoff

Short shelf-life code

When the UI changes:

1. Test fails ❌
2. Fix the selector ✅
3. Documentation updates automatically 📚
4. Accessibility re-audited ♿
5. Visual baseline regenerates 🎨

One fix, four outputs stay current

Invest in Your Testing Platform

> E2E tests are the *only* place where you have a real browser rendering real pages with real user interactions.

If you're already paying the cost of that context:

Extract every ounce of value from it.

One Codebase. Multiple Outputs.

That's the pitch.

Questions?

Resources:

- GitHub: `openedx-e2e-tests`
- Docs: `docs/VISUAL_REGRESSION.md`
- Testdoc: `utils/testdoc.ts`
- Visual Regression: `tests/common/visual-regression-helpers.ts`